

DOCUMENTO DE ARQUITECTURA DE SOFTWARE

AlgoBlocks

(Proyecto DGS)

*Especificación Técnica del Stack, Estructura del Monorepo y Patrones de
Diseño Arquitectónicos*

Para uso de:	Cuerpo de Profesores y Colaboradores
Fecha de Creación:	Junio, 2026
Estado:	Aprobado - Configuración de Monorepo Lista

1. Introducción y Contexto del Proyecto

El proyecto AlgoBlocks (DGS) surge como una modernización educativa para la facultad. Anteriormente, los alumnos utilizaban una aplicación de escritorio para comprender los fundamentos de la lógica de programación. Dicho entorno se basaba en la construcción de algoritmos mediante diagramas de bloques (conceptualmente similares a los diagramas de flujo tradicionales, pero con una representación visual más amplia y acotada al flujo de ejecución).

La nueva propuesta migra esta lógica a una aplicación web moderna (SPA), permitiendo a los estudiantes acceder de manera ágil desde cualquier navegador sin necesidad de instalaciones locales complejas. La aplicación tiene como objetivo principal que el estudiante aprenda a componer e interpretar algoritmos, resolviendo desde problemas básicos (como contadores y acumuladores) hasta algoritmos avanzados (como la detección de números primos, transposición de matrices y operaciones con vectores).

Los 6 Bloques Lógicos Fundamentales

El estudiante compone sus soluciones arrastrando, apilando y anidando 6 elementos lógicos que representan las estructuras de datos y control básicas presentes en cualquier lenguaje de programación:

- **1. Input (Entrada):** Permite solicitar datos al usuario para guardarlos en una variable. Ej: `|input: x|` solicita un valor y lo asigna a la variable 'x'.
- **2. Output (Salida):** Muestra en pantalla un texto literal o el resultado de evaluar una variable/expresión. Ej: `|output: x|` muestra el contenido de x; `|output: "hola"|` imprime un texto.
- **3. Asignación:** Inicializa o actualiza el valor de una variable. Permite operaciones de acumulación. Ej: `|asigna: x = 5|` o `|asigna: c = c + 1|`.
- **4. Alternativa (Condicional):** Bloque doble que define una condición. Se expande visualmente hacia abajo mostrando dos caminos: el camino del verdadero (se ejecuta si la condición se cumple) y el camino del falso (se ejecuta en caso contrario).
- **5. Ciclo Mientras (While):** Bucle que repite continuamente la ejecución de los bloques que contiene en su cuerpo mientras la condición lógica evaluada continúe siendo verdadera.
- **6. Ciclo Para (For):** Bucle que repite un bloque de código un número fijo de veces, incrementando una variable de control desde un límite inferior (a) hasta un límite superior (b).

2. Estructura del Monorepo (algoblocks/)

Para facilitar la administración del proyecto, evitar problemas de sincronización de API y permitir que un único repositorio de Git contenga todo el ciclo de vida, adoptamos la estructura de Monorepo. Esto permite compartir documentación, tipados de datos, configuraciones de despliegue Docker y orquestar el entorno local de forma unificada.

La estructura actual de directorios en el repositorio es la siguiente:

```
algoblocks/
├── frontend/           # SPA interactiva en React + TypeScript + Vite
│   ├── src/           # Código fuente de componentes, intérprete y hooks
│   ├── public/        # Recursos estáticos del navegador
│   └── package.json    # Dependencias del cliente
├── backend/           # API Restful en Node.js + Express + TypeScript + Prisma
│   ├── src/           # Controladores, rutas y middlewares estructurados
│   ├── prisma/        # Esquema de base de datos e historial de migraciones
│   └── package.json    # Dependencias del servidor
├── docs/              # Carpeta central de documentación del proyecto
│   ├── arquitectura/  # Decisiones de diseño y este documento PDF
│   ├── casos_de_uso/  # Requerimientos de estudiantes y profesores
│   ├── diagramas/     # Diagramas UML y Entidad-Relación de base de datos
│   └── reuniones/     # Minutas de seguimiento
├── database/          # Volumen y scripts de inicialización SQL
├── docker/            # Dockerfiles de producción
├── docker-compose.yml  # Orquestación de BD Postgres y pgAdmin locales
└── .gitignore         # Exclusiones de Git a nivel monorepo
```

3. Patrón de Arquitectura en el Backend

Tradicionalmente, en la web se utilizaba el patrón MVC (Model-View-Controller) donde el servidor compilaba y retornaba las vistas visuales (ej. HTML mediante plantillas EJS o Pug). En las aplicaciones modernas, separamos la vista por completo. El backend de AlgoBlocks expone una API REST pura y adopta una Arquitectura en Capas (Layered Architecture).

Este enfoque promueve el desacoplamiento de código, la reutilización de lógica de negocio y facilita la ejecución de pruebas automatizadas al separar la capa de transporte HTTP de la base de datos.

Desglose de Capas en Backend

- **Rutas (Routes):** Capa exterior que intercepta las peticiones del protocolo HTTP y las deriva al controlador adecuado según el endpoint solicitado.
- **Controladores (Controllers):** Intermediarios encargados de desempaquetar la petición HTTP, validar los parámetros del cliente (como IDs o payloads JSON) y estructurar la respuesta HTTP (códigos 200, 400, 500) junto con la respuesta JSON.
- **Servicios (Services):** Capa que contiene la lógica de negocio real de la aplicación. Es completamente independiente de Express u otro framework HTTP. Por ejemplo, calcula si un alumno completó correctamente un desafío o valida las reglas de un docente.
- **Modelos / ORM (Prisma):** Interactúa directamente con PostgreSQL. Prisma genera tipos de TypeScript automáticos mapeados a las tablas (User, Project, Challenge, Progress), garantizando consistencia en tiempo de compilación.
- **Middlewares:** Interceptores que ejecutan tareas transversales antes de que la petición llegue al controlador, tales como la validación de tokens JWT de sesión o el manejo global de errores del servidor.

4. Patrón de Arquitectura en el Frontend

En el Frontend (React + TypeScript), implementamos una separación que asemeja el patrón MVVM (Model-View-ViewModel) o la arquitectura basada en Contenedores, Hooks y Motor de Ejecución. El objetivo es que la interfaz visual de bloques (la Vista) no contenga código complejo sobre cómo se interpretan y ejecutan dichos bloques.

Módulos Principales del Frontend

- **Tipos de Bloques (Types):** Definidos en `types/index.ts`, proveen una estructura estricta y recursiva. Un bloque `AlternativeBlock` contiene arreglos de `Block[]` en sus caminos verdaderos y falsos. Esto nos permite almacenar los diagramas en la base de datos de manera uniforme como un árbol JSON.
- **Intérprete (Execution Engine):** Codificado en `engine/interpreter.ts`, este motor recorre la estructura del programa secuencialmente. Al ser asíncrono (`async/await`), permite detener la ejecución cuando se encuentra un bloque 'input' para esperar a que el alumno escriba un dato en la consola web, reanudando la ejecución después. También evalúa expresiones de manera segura.
- **Estado Global (Store):** Manejado mediante Contexto o una biblioteca liviana (ej: Zustand). El store contiene la lista de bloques en el canvas del editor de diagramas, permitiendo arrastrar, clonar, conectar y eliminar bloques de forma global sin propagar props innecesarios.
- **Componentes Visuales (Components):** Divididos en comunes (botones, modales, barras) y específicos del editor (el Lienzo/Canvas, el menú de bloques, y los bloques visuales individuales). Son puramente visuales y reaccionan al estado del Store.

Definición de Tipos en TypeScript

Para entender la estructura de datos que representa los bloques lógicos, los definimos así:

```
export type BlockType = 'input' | 'output' | 'assignment' | 'alternative' | 'while' | 'for';

export interface BaseBlock {
  id: string;
  type: BlockType;
}

export interface InputBlock extends BaseBlock {
  type: 'input';
  variableName: string;
}

export interface AlternativeBlock extends BaseBlock {
  type: 'alternative';
  condition: string; // Ej: 'x > 4'
  trueBranch: Block[];
  falseBranch: Block[];
}
```